Maddie Masiello
Lab Group A1
EE 329-03 S'25
2025-Apr-7

Introduction:

This submission demonstrates the successful implementation of both parts of Assignment A1: a 4-bit binary LED counter and an instruction execution timing measurement system using the STM32L4 microcontroller. The LED counter correctly counts from 0 to 15 with a visible delay, and the oscilloscope was used to calibrate and measure the delay loop. Execution timing was measured for a range of data types and arithmetic operations, and results are summarized in Table A1(b).

All functionality operates as intended. No significant bugs were encountered. However, accurate oscilloscope triggering was sometimes challenging due to signal jitter during fast instruction cycles.

Key lessons from this lab include the importance of precise timing in embedded systems, how to configure GPIO outputs and use them for measurement, and how different data types and operations impact execution time. Future improvements could include automating instruction swaps with macros or typedefs and using hardware timers instead of software loops for more accurate delays.

Link to YouTube Video Presentation: https://youtu.be/mrWB2fJikeg

(Date of posted video: 04/09/2025)

(File size: 850.4MB)

STM32L4A6 Wiring Diagram for A1:

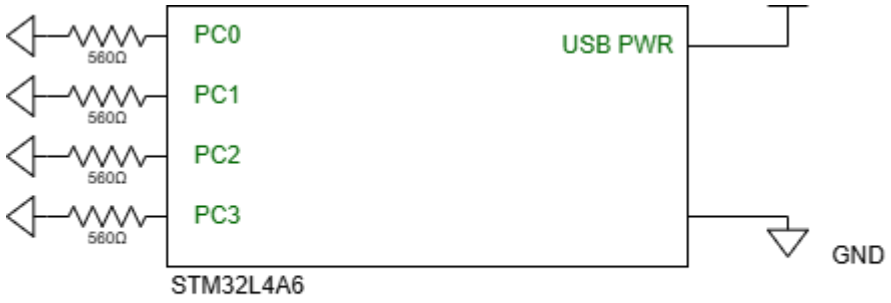

Figure 1. STM32L4A6 Wiring Diagram for A1

Table A1(a): LED circuit calculations & measurements for R = 560 Ω

LED drive circuit	GPIO Vout	LED Vf	LED current
calculations	3.3 - 0.4 = 2.9V	1.8V	2mA
measurements	3.1V	1.82V	1.592mA

Table A1(b): instruction execution timing for different data types

Execution Time	uint8_t (3)	int32_t (1000000000)	int64_t (1000000000000ULL)	Float (3.14f)	Double (3.141)
function call latency	255 ns	498 ns	1000 us	507 ns	1.00 us
test_var = num	2.01 us	2.492 us	2.25 us	2.00 us	2.54 us
test_var = num + 1	2.50 us	2.75 us	4.01 us	3.75 us	3.99 us
test_var = num * 3	2.51 us	2.73 us	4.763 us	3.75 us	3.99 us
test_var = num / 3	2.02 us	2.49 us	3.75 us	3.75 us	3.99 us
test_var = num * num	2.50 us	2.75 us	4.00 us	3.75 us	3.75 us
test_var = num % 10	4.52 us	6.76 us	3.768us	N/A	N/A
test_var = pow(num, 3)	N/A	4.00 us	3.75 us	3.51us	4.25 us
test_var = sqrt(num)	N/A	3.96 us	3.75 us	3.5 us	3.76 us
test_var = sin(num)	N/A	3.96 us	3.75 us	3.5 us	3.74 us

Appendix:

Formatted Source Code:

```
------------------------------------------------------------------------
main.h
------------------------------------------------------------------------
/*******************************************************************************
 * EE 329 A1 LED COUNTER AND EXECUTION TIMER
 ******************************************************************************
 * @file           : main.c
 * @brief          : Implements a binary LED counter and measures instruction
 *                   execution timing using GPIO pins.
 * project         : EE 329 S'23 Assignment 1
 * authors         : Maddie Masiello - mmasiell@calpoly.edu
 * version         : 1.0
 * date            : 04/7/2025
 * compiler        : STM32CubeIDE v.1.12.0 Build: 14980_20230301_1550 (UTC)
 * target          : NUCLEO-L4A6ZG
 * clocks          : 4 MHz MSI to AHB2
 * @attention      : (c) 2025 Maddie Masiello. All rights reserved.
```

```
******************************************************************************
 * PROJECT PLAN:
 * 1. LED Counter:
 *    - Connect 4 LEDs to GPIO pins PC0, PC1, PC2, and PC3.
 *    - Implement a binary counter that counts from 0 to 15 and displays the
 *      count on the LEDs with a calibrated software delay.
 *    - Measure GPIO output voltage, LED voltage, and LED ON current.
 *
 * 2. Execution Timing:
 *    - Measure the execution time of a function call and an arithmetic
 *      instruction using GPIO pins PC0 and PC1.
 *    - Modify the TestFunction() to test various arithmetic operations.
 *    - Use an oscilloscope or logic analyzer to capture timing pulses.

******************************************************************************
 * GPIO CONFIGURATION:
 * - PC0–PC3: Output mode, push-pull, no pull-up/pull-down, high speed.
 * - PC0: Toggles before and after TestFunction() call.
 * - PC1: Toggles before and after arithmetic instruction inside
TestFunction().

******************************************************************************
 * REVISION HISTORY:
 * 1.0 04/7/2025 Maddie Masiello - Initial implementation.

******************************************************************************
 * TODO:
 * - Immplement more accurate timing methods

******************************************************************************
 * 45678-1-2345678-2-2345678-3-2345678-4-2345678-5-2345678-6-2345678-7-234567
*/
/* USER CODE END Header */

#include "main.h"
#include <math.h>

typedef uint8_t var_type; // Defines var_type as uint32_t for 8-bit variable
type
//typedef uint32_t var_type;
//typedef uint64_t var_type;
//typedef float var_type;
```

```c
//typedef double var_type;

void SystemClock_Config(void);
uint8_t TestFunction(uint8_t num);
void delay_loop(volatile int count);

/* ----------------------------------------------------------------------------
-
 * function : main( )
 * INs      : void
 * OUTs     : main_var (result of TestFunction), LED output on PC0–PC3
 * action   : initializes GPIO, runs LED counter, and measures execution time
 *
 * authors  : Maddie Masiello - mmasiell@calpoly.edu
 * version  : 1
 * date     : 04/7/2025
 * ----------------------------------------------------------------------------
*/

int main(void)
{
  volatile uint8_t main_var;
  //volatile uint32_t main_var;
  //volatile uint64_t main_var;
  //volatile float main_var;
  //volatile double main_var;

  int iteration_count = 0; // initialization of iteration count
  HAL_Init();
  SystemClock_Config();

  RCC->AHB2ENR |= (RCC_AHB2ENR_GPIOCEN);
  GPIOC->MODER &= ~(GPIO_MODER_MODE0 | GPIO_MODER_MODE1 |
                    GPIO_MODER_MODE2 | GPIO_MODER_MODE3);
  GPIOC->MODER |= (GPIO_MODER_MODE0_0 | GPIO_MODER_MODE1_0 |
                   GPIO_MODER_MODE2_0 | GPIO_MODER_MODE3_0);
  GPIOC->OTYPER &= ~(GPIO_OTYPER_OT0 | GPIO_OTYPER_OT1 |
                     GPIO_OTYPER_OT2 | GPIO_OTYPER_OT3);
  GPIOC->PUPDR &= ~(GPIO_PUPDR_PUPD0 | GPIO_PUPDR_PUPD1 |
                    GPIO_PUPDR_PUPD2 | GPIO_PUPDR_PUPD3);
  GPIOC->OSPEEDR |= ((3 << GPIO_OSPEEDR_OSPEED0_Pos) |
                     (3 << GPIO_OSPEEDR_OSPEED1_Pos) |
                     (3 << GPIO_OSPEEDR_OSPEED2_Pos) |
```

```
                      (3 << GPIO_OSPEEDR_OSPEED3_Pos));
  GPIOC->BRR = (GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3);


  GPIOC->BSRR = (GPIO_PIN_0);           // turn on PC0
  main_var = TestFunction(100000000);   // call test function
  GPIOC->BRR = (GPIO_PIN_0);            // turn off PC0

  // repeat the LED counter 3 times, count from 0 to 15 in binary on LEDS
  for (iteration_count = 0; iteration_count < 3; iteration_count++)
  {
    for (int led_count = 0; led_count < 16; led_count++)
    {
      GPIOC->BRR = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3;
      GPIOC->BSRR = (led_count & 0x0F); // Binary output on LED's
      delay_loop(180000);               // Tuned delay for LED visibility
    }
  }

  // infinite loop to repeat the function call and measure execution time
  while (1)
  {
    GPIOC->BSRR = GPIO_PIN_0;   // set PC0 high to start timing

    main_var = TestFunction(3);                    // for uint8_t
    //main_var = TestFunction(1000000000);         // for uint32_t
    //main_var = TestFunction(1000000000000ULL);   //for unt32_t
    //main_var = TestFunction(3.14f);              // for float
    //main_var = TestFunction(3.141);              // for double

    GPIOC->BRR = GPIO_PIN_0;                    // set PC0 low to stop timing
  }
}

//blocking delay loop
void delay_loop(volatile int count) {
  while (count--);
}

/* ---------------------------------------------------------------------------
-
 * function : TestFunction( )
```

```
 * INs      : num
 * OUTs     : test_var
 * action   : toggles PC1 and tests various arithmetic operations
 * authors  : Maddie Masiello - mmasiell@calpoly.edu
 * version  : 1
 * date     : 04/7/2025
 * ------------------------------------------------------------------------
*/

 uint8_t TestFunction(uint8_t num)
{
  //change type as needed
  volatile uint8_t test_var;



  GPIOC->BSRR = (GPIO_PIN_1);           // turn on PC1

  //change test functions as needed for testing
  test_var = num;
  //test_var = num + 1;
  //test_var = num * 3;
  //test_var = num / 3;
  //test_var = num * num;
  //test_var = num % 10;
  //test_var = pow(num, 3)
  //test_var = sqrt(num);
  //test_var = sin(num);

  GPIOC->BRR = (GPIO_PIN_1);           // turn off PC1 (low)

  return test_var;
}

void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Configure the main internal regulator output voltage*/
  if (HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1) != HAL_OK)
  {
    Error_Handler();
```

```
  }

  /** Initializes the RCC Oscillators according to the specified parameters
   * in the RCC_OscInitTypeDef structure.
   */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_MSI;
  RCC_OscInitStruct.MSIState = RCC_MSI_ON;
  RCC_OscInitStruct.MSICalibrationValue = 0;
  RCC_OscInitStruct.MSIClockRange = RCC_MSIRANGE_6;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
   */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYSCLK
                              | RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_MSI;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
  {
    Error_Handler();
  }
}

/* USER CODE BEGIN 4 */

/* USER CODE END 4 */

/**
 * @brief  This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL
    error return state */
```

```
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
```